Chapter 27: Sorting Networks

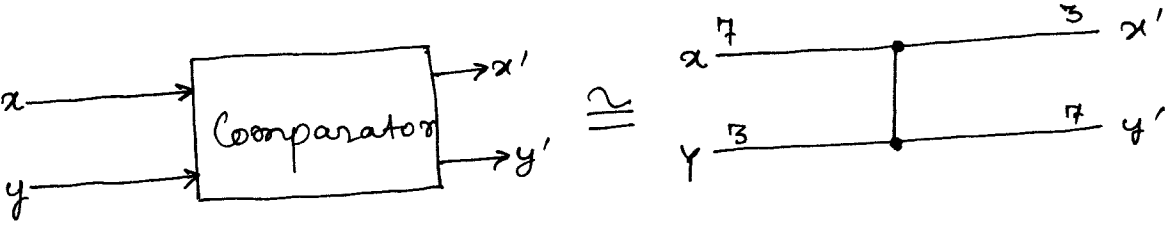
1) Comparison network model of computation differs from the random-access machines model in two important respects :-

(i) in a RAM model, operations occur serially, whereas operations in a comparison network occur in parallel.

(ii) comparison networks can only perform comparisons. Thus, an algorithm such as counting sort cannot be implemented on a comparison network.

2)

→ **Comparator:** A comparator has 2 inputs which it outputs in a sorted order. Inputs appear on the left and outputs appear on the right, with the smaller input value appearing as the top output and the larger input value appearing as the bottom output.



$$x' = \min(x, y)$$
$$y' = \max(x, y)$$

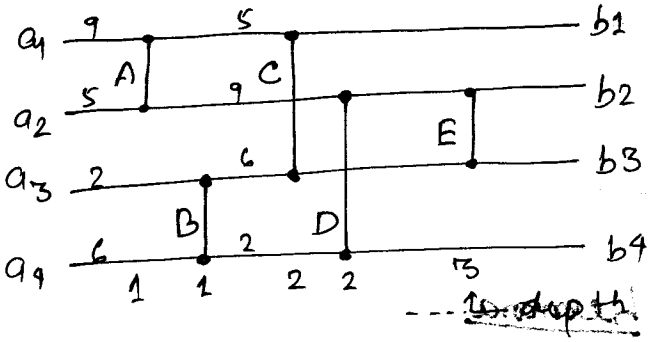
We assume that a comparator operates in $O(1)$ time, i.e. the time lag between the appearance of input values and the production of output values is a constant.

→ **Comparison network:** A comparison network is a set of comparators interconnected by wires. A comparison network on n inputs is drawn as a collection of n horizontal lines with comparators stretched vertically.

The main requirement for interconnecting comparators is that the graph of interconnections must be acyclic. Data should move from left to right. If we trace a path from the O/P of one comparator to the I/P of another comparator and to the O/P of some other comparator and so forth, the path we trace must never cycle back on itself and include the same comparator twice.

→ Each comparator produces its O/P values only when both of its input values are available to it.

Assuming each comparator takes unit time to compute its O/P values and supposing that the I/P $\langle a_1, a_2, a_3, a_4 \rangle = \langle 9, 5, 2, 6 \rangle$ is available at time 0, comparators A and B produce their O/P at time 1.



I/Ps to C and D are available at time 1 and they produce O/P at time 2. E produces O/P at time 3.

→ The depth of an I/P wire of a comparison network is defined to be 0. If the depth of the I/P wires of a comparator is d_x and d_y, the O/P wires

would have depth $\max(d_x, d_y) + 1$. The depth of a comparator is said to be the depth of its O/P wires. The depth of a comparison network is the maximum depth of any O/P wire = the maximum depth of any comparator.

→ Running time of a comparison network is the time it takes for all O/P wires to receive their values once the I/P wires receive theirs. Under the assumption that each comparator takes unit time, and the the I/P is available at time 0, a comparator at depth d produces its O/P at time d. Thus the running time of a comparison network is equal to the depth of the comparison network = the max. no. of comparators any input element can pass through as it travels from an I/P wire to an O/P wire.

→ A sorting network is a comparison network that sorts the I/P sequence to produce a monotonically increasing sequence of values as output.

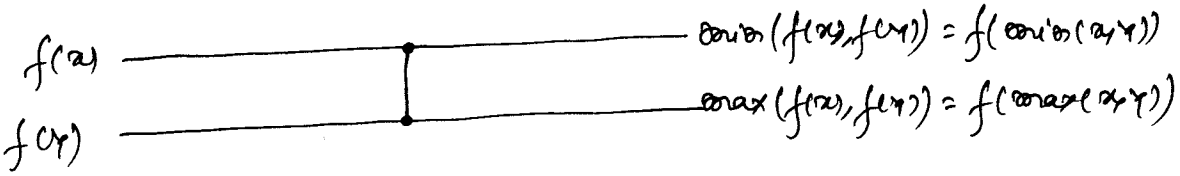
3) The zero-one principle:

Lemma

If a comparison network transforms the I/P sequence $a = \langle a_1, a_2 \ldots a_n \rangle$ into the O/P sequence $b = \langle b_1, b_2 \ldots b_n \rangle$, then for any monotonically increasing function f, the network transforms the I/P sequence $f(a) = \langle f(a_1), f(a_2), \ldots f(a_n) \rangle$ into the O/P sequence $f(b) = \langle f(b_1), f(b_2), \ldots f(b_n) \rangle$.

Proof

If f is a monotonically increasing function and $f(x)$, $f(y)$ be the inputs to a comparator the outputs of the comparator will be $\max(f(x), f(y)) = f(\max(x,y))$ and $\min(f(x), f(y)) = f(\min(x,y))$.

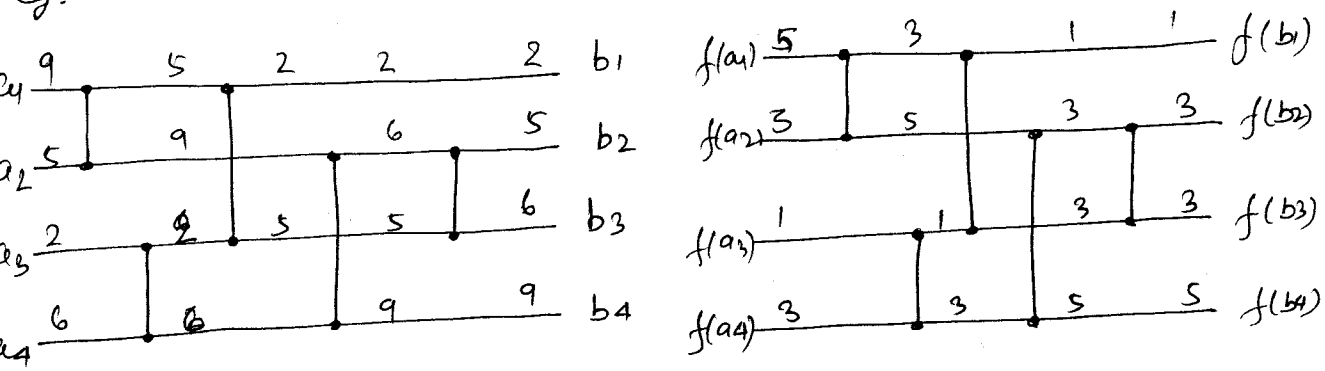


We use induction on the depth of each wire to prove a stronger result than the statement of the lemma: if some wire (may or may not be an O/P wire) assumes the value a_i when I/P sequence a is applied to the network, then it will assume the value $f(a_i)$ when I/P sequence $f(a)$ is applied. Because the output wires are included in this statement, proving it will prove the lemma.

For the basis, consider a wire at depth 0, if a_i is applied to this wire as the sequence a is applied to the network then the result follows trivially: if $f(a)$ is applied to the network, the wire carries $f(a_i)$.
For the inductive step, consider a wire that is the O/P of some comparator at depth d. Depth of the wire would also be d. The input wires to the comparator would be at depth strictly less than d. Inductive hypothesis implies that if the I/P wires carry values a_i and a_j when the I/P sequence a is applied to them, they would carry values $f(a_i)$ and $f(a_j)$ when the I/P sequence $f(a)$ is applied to the comparison network.

The claim in the 1st part of the proof implies that the O/P wires of the comparator would carry $f(\min(a_i, a_j))$ and $f(\max(a_i, a_j))$. Since they carry $\min(a_i, a_j)$ and $\max(a_i, a_j)$ when the I/P sequence is a, the lemma is proved.

eg:-



$$f(x) = \lceil x/2 \rceil$$

Theorem (Zero-One principle)

If a comparison network with n inputs sorts all 2^n possible sequences of 0's and 1's correctly, then it sorts all sequences of arbitrary numbers correctly.

Proof: Suppose for the purpose of contradiction that there exists a sequence of arbitrary numbers that the network does not correctly sort. That is, there exists an I/P sequence $\langle a_1, a_2, \ldots, a_n \rangle$ containing elements a_i and a_j such that $a_i < a_j$, but the network places a_j before a_i in the O/P sequence.

We define a monotonically increasing function f as:-

$$f(x) = \begin{cases} 0 & \text{if } x \le a_i \\ 1 & \text{if } x > a_i \end{cases}$$

Thus $f(a_i) = 0$ and $f(a_j) = 1$.

Since the network places a_j before a_i in the o/p sequence, the above lemma implies that $f(a_j)$ should appear before $f(a_i)$ in the o/p sequence. Thus the network also fails to correctly sort the zero-one sequence $\langle f(a_1), f(a_2) \ldots f(a_n) \rangle$ which is a contradiction to the assumption that only one such sequence exists.

→ Once we have constructed a sorting network and proved that it can sort all zero-one sequences, then can use the zero-one principle to show that the network can properly sort sequences of arbitrary values.

4) A bitonic sorting network

→ **Bitonic sequence:** A sequence that monotonically increases and then monotonically decreases or can be circularly shifted to become monotonically increasing and then monotonically decreasing is called a bitonic sequence. eg: $\langle 1,4,6,8,3,2 \rangle$, $\langle 6,9,4,2,3,5 \rangle$, $\langle 9,8,3,2,4,6 \rangle$.

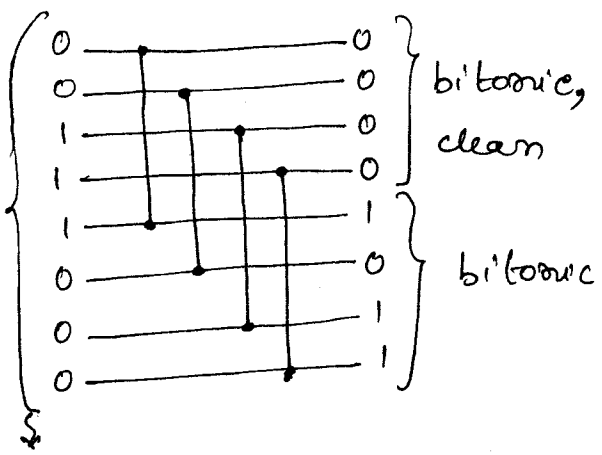
As a boundary condition, we say that any sequence of just 1 or 2 numbers is bitonic.
Zero-one sequences that are bitonic either have the form $0^i 1^j 0^k$ or $1^i 0^j 1^k$ where $i, j, k \ge 0$.
A sequence that is either monotonically increasing or monotonically decreasing is also 6

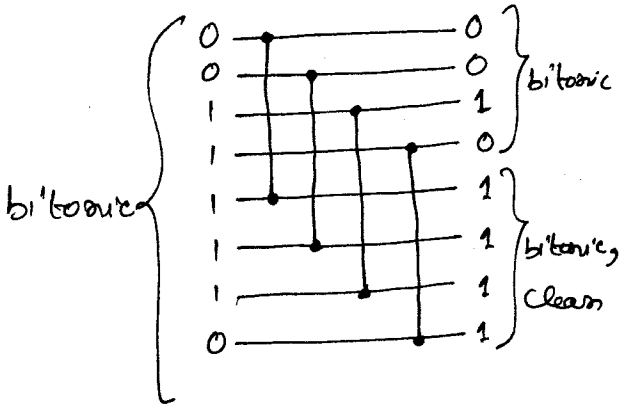
bitonic.

→ A comparison network that can sort bitonic sequences is called a bitonic sorter.

Half-cleaner:

A half-cleaner is a comparison network of depth 1 in which input line i is compared with input line $i+n/2$ for $i = 1, 2, \ldots, n/2$. (We assume n is even).





HALF-CLEANER[8]: half-cleaner with 8 I/Ps and 8 O/Ps.

→ When a bitonic sequence of 0's and 1's is applied as input to a half-cleaner, the half-cleaner produces an O/P sequence in which smaller values are in the top half, larger values are in the bottom half and both halves are bitonic.

At least one of the halves is clean - consisting of either all 0's or all 1's. All clean sequences are bitonic.

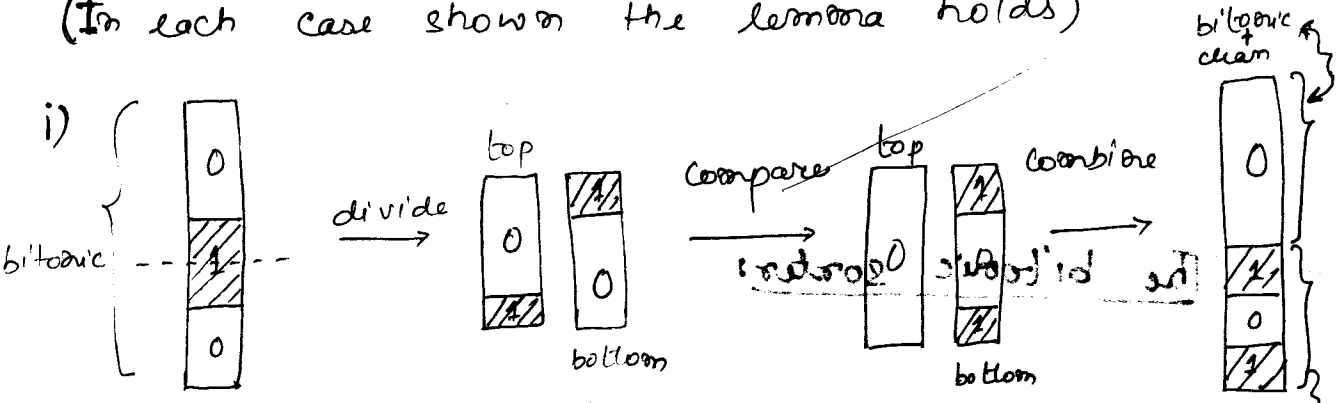
Lemma

If the input to a half-cleaner is a bitonic sequence of 0's and 1's, then the O/P satisfies the following properties: both the top half and the bottom half are bitonic, every element of the top half is at least as small as every element of the bottom half, and at least one half is clean.

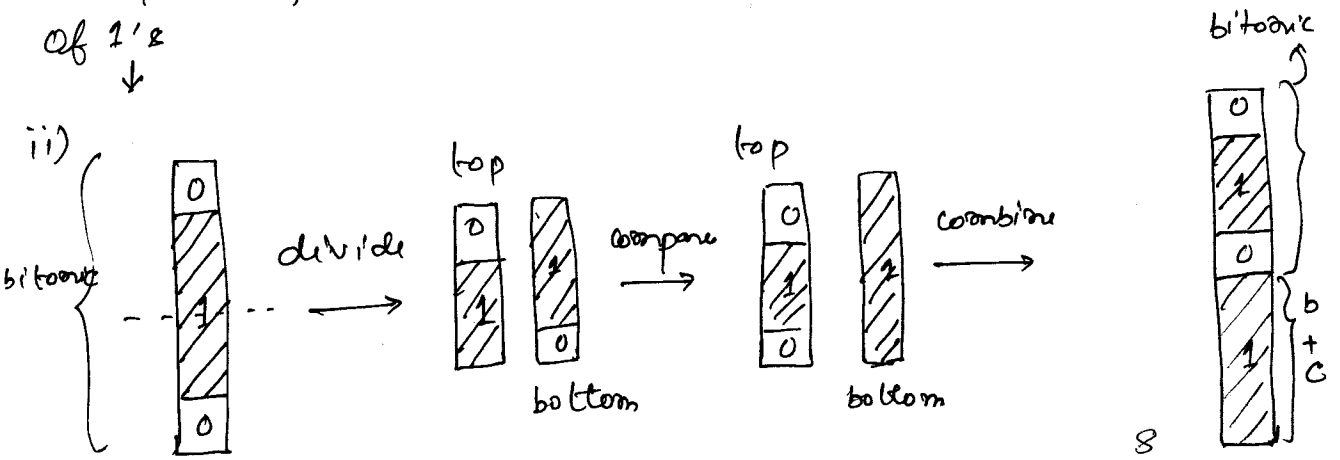
Proof: Without loss of generality, suppose the I/P is of the form $0^i 1^j 0^k$ (the situation in which the input is of the form $1^i 0^j 1^k$ is symmetric).

Depending upon where the midpoint $n/2$ falls there are the following cases:—

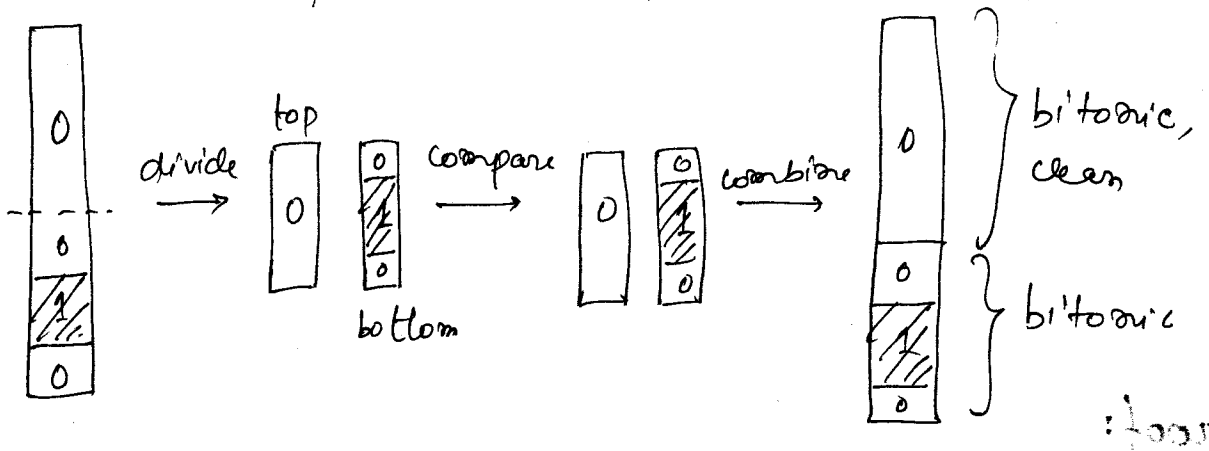
(In each case shown the lemma holds)



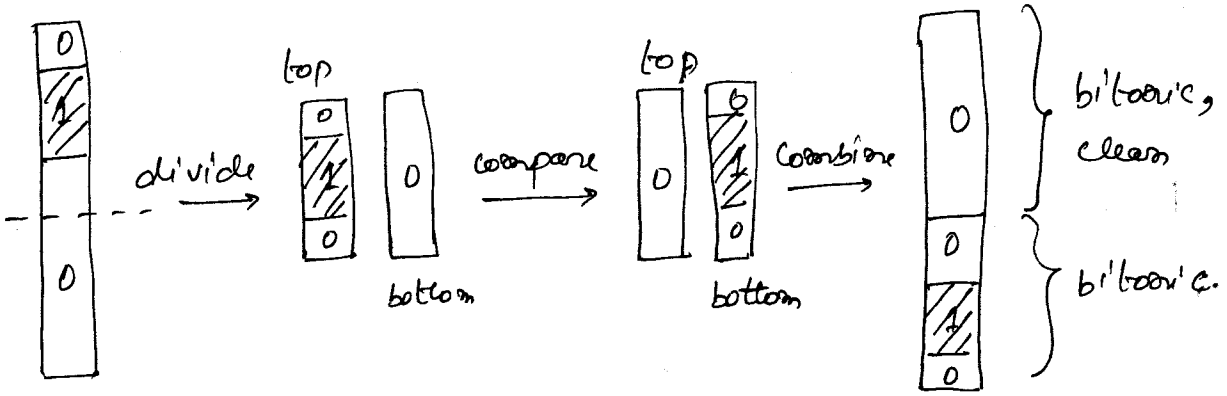
↑ midpoint falls in the middle subsequences of 1's



iii) midpoint falls in the top subsequence of 0's :-



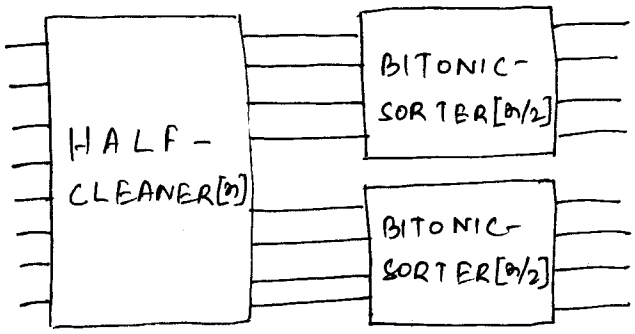
iv) midpoint falls in the bottom subsequence of 0's:-



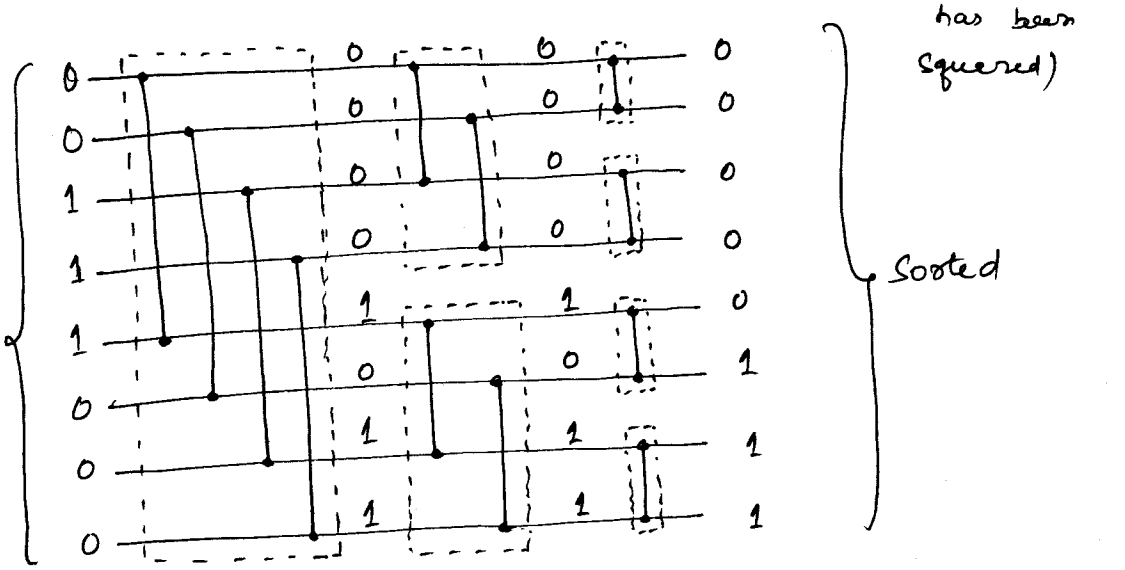
→ The bitonic sorter:

A network that sorts bitonic sequences is called a bitonic sorter. A BITONIC-SORTER[n] is comprised of lg n stages such that every stage converts its bitonic I/P sequence into 2 bitonic O/P sequences of half the I/P size such that every element in the top half is at least as small as every element in the bottom half. every a single HALF-CLEANER of

A recursive depiction of BITONIC-SORTER[n] is as follows:-



The network after unrolling the recursion: (Each half-cleaner has been squeezed)



→ The depth $D(n)$ of BITONIC_SORTER[n] is given by the recurrence :—

$$D(n) = \begin{cases} 0 & \text{, if } n = 1 \\ D(n/2) + 1 & \text{, if } n = 2^k \text{ and } k \geq 1 \end{cases}$$

whose solution is $D(n) = \lg n$.

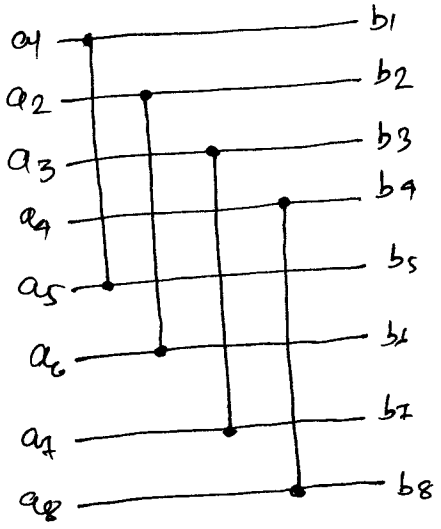
A zero-one bitonic sequence can be sorted by a BITONIC-SORTER. The zero-one principle then implies that any bitonic sequence of arbitrary numbers can then be sorted by this network.

5) Merging Network

→ A network that can merge two sorted input sequences into one sorted output sequence is called a merging network.

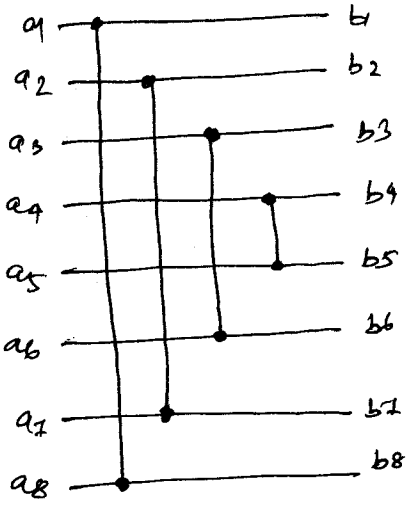
→ Given two sorted sequences, if we reverse the order of the second sequence and then concatenate the two sequences, the resulting sequence is bitonic. For example, given the zero-one sequences $X = 00000111$ and $Y = 00001111$, we reverse Y to get $Y^R = 11110000$. Concatenating X and Y^R yields 00000111111110000, which is bitonic. Feeding XY^R as the I/P to some bitonic sorter will give the correct sorted O/P.

→ We can perform the above operations if we modify the BITONIC-SORTER to implicitly reverse the 2nd half of its I/P. Thus, if we modify the first half-cleaner of BITONIC-SORTER to compare the input lines i and $n-i+1$ instead of comparing i and $n/2+i$ for $i = 1, 2, \ldots, n/2$ we are done.

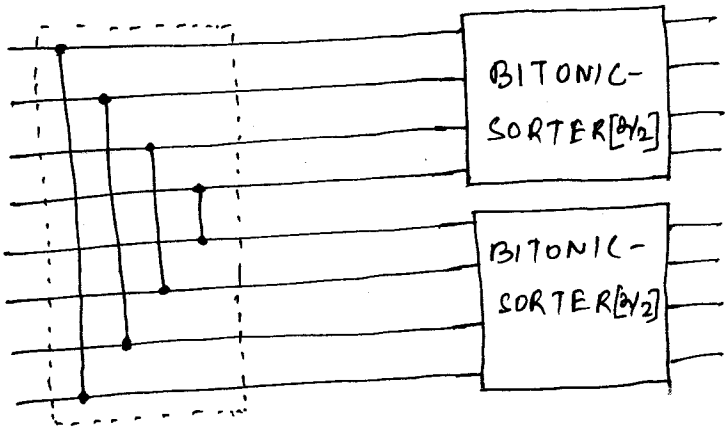


Original half-cleaner

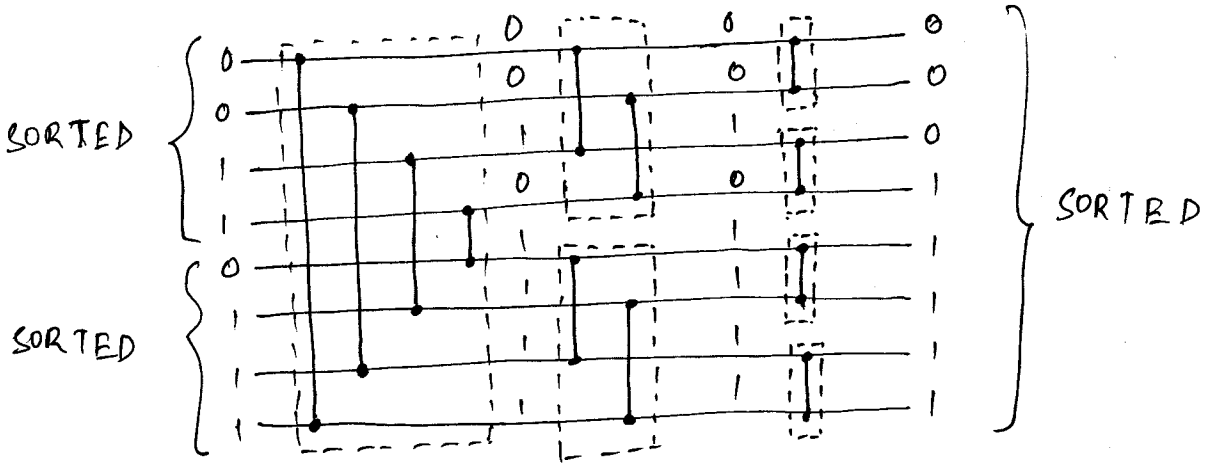
Modified half-cleaner



→ Recursive definition of MERGER[n] :-



Recursion unrolled. MERGER[8]:-

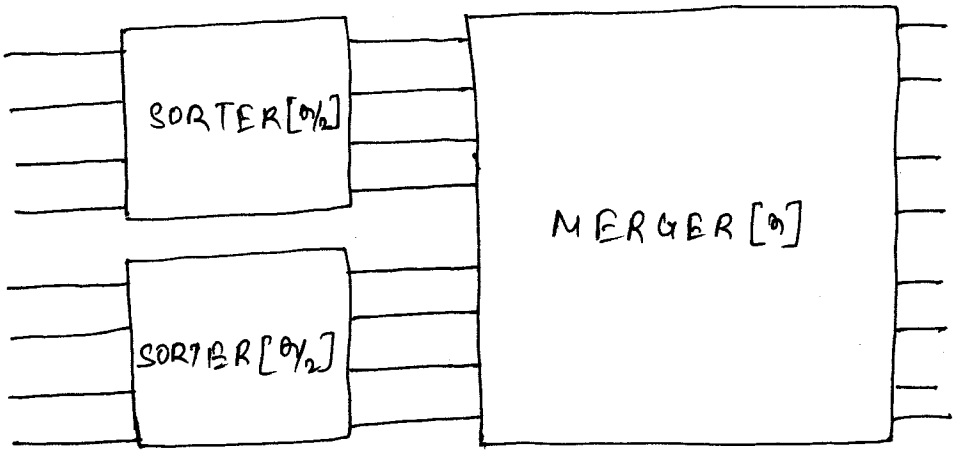


→ The depth of MERGER[n] is lg n, the same as that of BITONIC-SORTER[n], since only the first stage of MERGER[n] is different from BITONIC-SORTER[n].

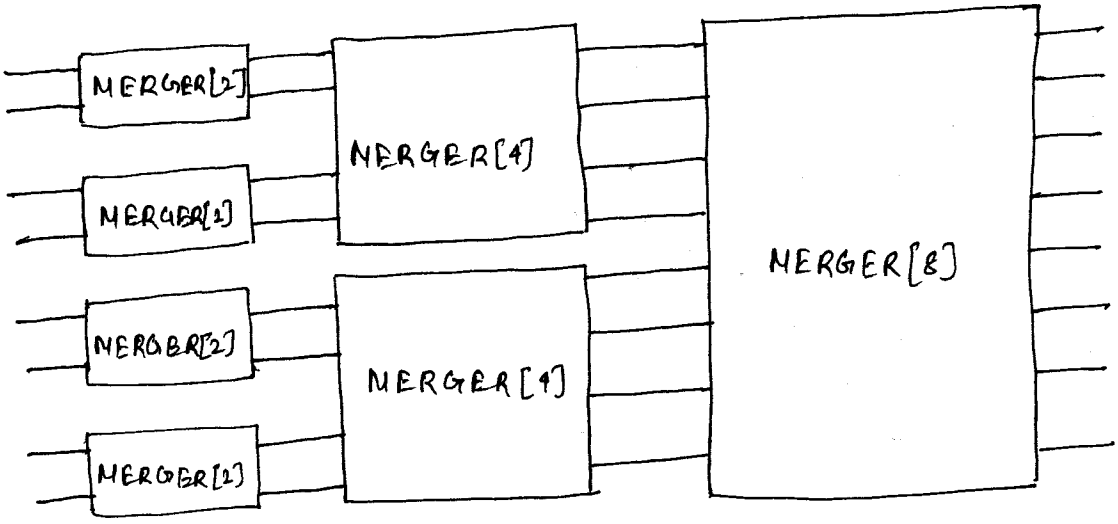
c) A sorting network

The sorting network SORTER[n] uses merging networks to implement a parallel version of merge sort.

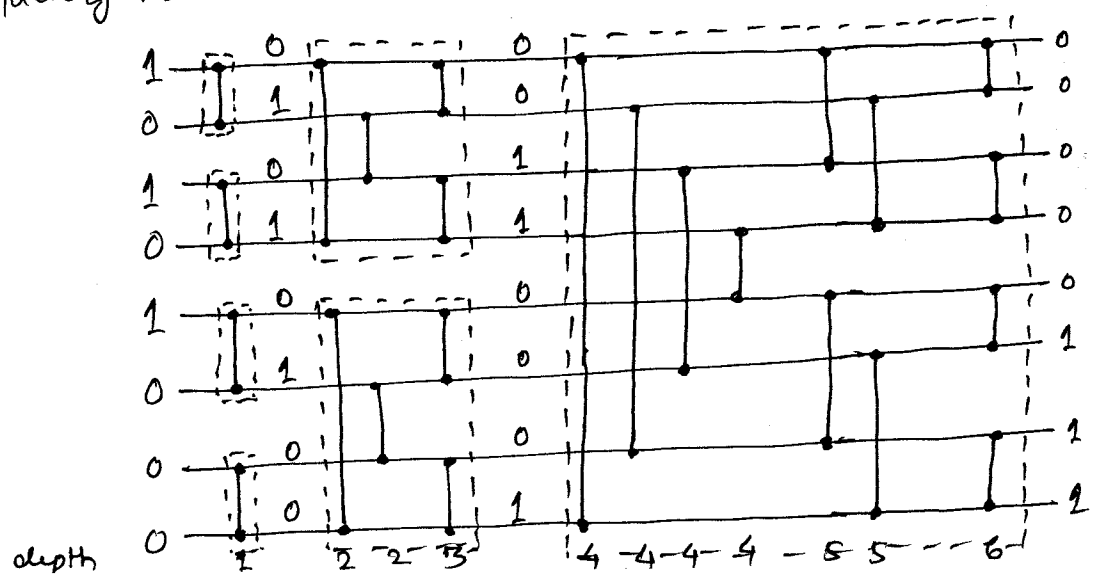
→ A recursive construction of SORTER[n]:-



→ Unrolling the recursion:—



→ Replacing the MERGER boxes with the actual sorting networks



→ The Depth $D(n)$ of SORTER[n] is the depth $D(n/2)$ of SORTER [$n/2$] plus the depth $\lg n$ of MERGER[n]. Consequently, the depth of SORTER[n] is given by the recurrence :-

$$D(n) = \begin{cases} 0 & , \text{if } n = 1 \\ D(n/2) + \lg n & , \text{if } n = 2^k \text{ and } k \geq 1 \end{cases}$$

whose solution is $D(n) = \Theta(\lg^2 n)$. Thus, we can sort n numbers in parallel in $O(\lg^2 n)$ time.